FirstOffer

DATA ANALYST INTERVIEW KIT · SQL

20 SQL Questions Every Data Analyst Should Practise

The SQL questions interviewers use to test analysts — joins, ranking, window functions, duplicates and business metrics — each with a tested query and the exact output it returns.

Basics | Joins | Ranking & Top-N | Window functions | Data quality

20 questions

19 tested examples

5 topics

firstoffer.online

Fresher jobs · Direct apply links · No middlemen

Written from the interviewer's side · Easy → Medium

START HERE

How to use this kit

1
Read the question and answer it out loud before you look at the model answer.

2
Read “What the interviewer is checking” — that's what you're actually scored on.

3
Run the code yourself, change one thing, and predict the new output before running it again.

SPOT THE CLUE

If the question says… reach for…

“For each …” / “per …”	**GROUP BY**
“… but only groups where the total …”	**HAVING**
“Never / missing / not in the other table”	**LEFT JOIN … WHERE right.id IS NULL**
“Compare a row with another row of the same table”	**Self-join**
“Top N / Nth highest / per department”	**DENSE_RANK() OVER (PARTITION BY …)**
“Running total / previous month / % change”	**SUM() OVER · LAG()**
“Share of total”	**SUM(x) / SUM(SUM(x)) OVER ()**
“Buckets / bands / flags”	**CASE WHEN**

IN THE INTERVIEW ROOM

1 · Ask about the tables & grain **2 · Clarify ties & NULLs** **3 · Write it step by step (CTEs)**
4 · Run it on a tiny example **5 · Explain the result**

SAMPLE DATA

The data used in this kit

Every query in this kit was run on these four small tables, and the result shown under each query is the real output.

employees

emp_id	name	dept_id	manager_id	salary	hire_date
1	Asha	1	NULL	150000	2019-04-01
2	Rahul	1	1	90000	2021-06-15
3	Priya	1	1	160000	2020-01-10
4	Karan	2	NULL	120000	2018-09-01
5	Sneha	2	4	70000	2022-03-01
6	Vikram	2	4	70000	2022-07-20
7	Meera	3	NULL	65000	2021-11-05
8	Arjun	1	1	90000	2023-02-14
9	Divya	3	7	48000	2024-01-08

departments

dept_id	dept_name
1	Engineering
2	Sales
3	HR
4	Finance

customers

customer_id	name	city
101	Ravi	Bengaluru
102	Neha	Pune
103	Imran	Delhi
104	Pooja	Bengaluru
105	Sameer	Mumbai

orders

order_id	customer_id	order_date	amount
1	101	2026-01-05	1200
2	102	2026-01-18	800
3	101	2026-02-02	1500
4	103	2026-02-11	400
5	101	2026-03-09	700
6	104	2026-03-15	2200
7	102	2026-03-28	950
8	103	2026-03-30	400

CONTENTS

20 questions, easy to medium

Warm-up · Basics every analyst must know

01	What is the difference between WHERE and HAVING?	Easy
02	COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)?	Easy
03	How do you handle NULLs? Why doesn't `manager_id = NULL` work?	Easy
04	UNION vs UNION ALL?	Easy
05	In what order does SQL actually execute a query?	Easy

Joins

06	Explain INNER, LEFT, RIGHT and FULL OUTER JOIN.	Easy
07	Find customers who have never placed an order.	Easy
08	Find employees who earn more than their manager.	Medium

Ranking & Top-N

09	Find the second highest salary.	Easy
10	Find the Nth highest salary (say N = 3).	Medium
11	ROW_NUMBER vs RANK vs DENSE_RANK?	Easy
12	Get the top 2 earners in each department.	Medium
13	Show the highest-paid employee in every department.	Medium

Data quality

14	How do you find duplicate records?	Easy
15	Remove duplicates but keep one copy of each.	Medium

Analytics with window functions

16	Calculate a running total of sales by date.	Medium
17	Show month-over-month revenue change.	Medium
18	What share of total revenue does each customer contribute?	Medium
19	Find customers whose average order value is above the overall average.	Medium
20	Group employees into salary bands and count each band.	Easy

Warm-up · Basics every analyst must know

01 What is the difference between WHERE and HAVING?

Filtering Easy

WHAT THE INTERVIEWER IS CHECKING

Do you know WHERE filters rows before grouping and HAVING filters groups after aggregation?

MODEL ANSWER

- WHERE filters individual rows before GROUP BY runs — it can't use aggregates like COUNT() or SUM().
- HAVING filters the groups produced by GROUP BY — that's where aggregate conditions go.
- Example: departments with at least 2 employees hired from 2020 onwards.

```sql
SELECT dept_id, COUNT(*) AS headcount
FROM employees
WHERE hire_date >= '2020-01-01'      -- row filter
GROUP BY dept_id
HAVING COUNT(*) >= 2;                -- group filter
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

dept_id	headcount
1	3
2	2
3	2

Common mistake Writing WHERE COUNT(*) > 2 — aggregates aren't allowed in WHERE.

Interviewer may ask next Can you use a column alias from SELECT inside HAVING? (Depends on the database — safer to repeat the expression.)

02 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)?

Aggregation Easy

WHAT THE INTERVIEWER IS CHECKING

Understanding of how NULLs and duplicates affect counts.

MODEL ANSWER

- COUNT(*) counts every row, including rows with NULLs.
- COUNT(column) skips rows where that column is NULL.
- COUNT(DISTINCT column) counts unique non-NULL values.

```sql
SELECT COUNT(*)                  AS all_rows,
       COUNT(manager_id)         AS has_manager,
       COUNT(DISTINCT manager_id) AS distinct_managers
FROM employees;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

all_rows	has_manager	distinct_managers
9	6	3

Common mistake Assuming COUNT(column) equals COUNT(*) on a column that has NULLs.

Interviewer may ask next How would you count NULLs in a column? (COUNT(*) - COUNT(col), or SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END).)

03 How do you handle NULLs? Why doesn't `manager_id = NULL` work?

NULLs Easy

WHAT THE INTERVIEWER IS CHECKING

NULL means 'unknown', so comparisons with = return unknown, not true.

MODEL ANSWER

- Use IS NULL / IS NOT NULL — any comparison with NULL using = or <> is never true.
- COALESCE(col, fallback) returns the first non-NULL value — handy for reports.
- Aggregates like SUM and AVG ignore NULLs.

```sql
SELECT name,
       COALESCE(CAST(manager_id AS TEXT), 'No manager') AS reports_to
FROM employees
WHERE manager_id IS NULL;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

name	reports_to
Asha	No manager
Karan	No manager
Meera	No manager

Common mistake WHERE manager_id = NULL returns zero rows without any error.

Interviewer may ask next What does AVG(col) do with NULLs vs 0s? (NULLs are skipped; zeros pull the average down.)

04 UNION vs UNION ALL?

Set operations · Easy

WHAT THE INTERVIEWER IS CHECKING

Duplicates and performance.

MODEL ANSWER

- UNION combines results and removes duplicate rows (needs an extra sort/dedupe step).
- UNION ALL keeps every row, duplicates included — faster when you know there are no duplicates or you want them.
- Both need the same number of columns with compatible types.

```sql
SELECT city FROM customers WHERE city = 'Bengaluru'
UNION ALL
SELECT city FROM customers WHERE city IN ('Bengaluru', 'Pune');
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

city
Bengaluru
Bengaluru
Bengaluru
Pune
Bengaluru

Common mistake Using UNION by default and silently losing legitimate duplicate rows.

Interviewer may ask next Change it to UNION — how many rows now? (2: Bengaluru, Pune.)

05 In what order does SQL actually execute a query?

Concepts Easy

WHAT THE INTERVIEWER IS CHECKING

Explains alias and filtering errors — a favourite fresher question.

MODEL ANSWER

- FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
- That's why a SELECT alias can't be used in WHERE (WHERE runs first) but usually can be used in ORDER BY.

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

Common mistake Saying SELECT runs first because it's written first.

Interviewer may ask next Why can't a window function like ROW_NUMBER() be used in WHERE? (Window functions are computed in the SELECT step — wrap it in a subquery/CTE.)

Joins

06 Explain INNER, LEFT, RIGHT and FULL OUTER JOIN.

Joins Easy

WHAT THE INTERVIEWER IS CHECKING

Can you predict which rows survive each join?

MODEL ANSWER

- INNER JOIN: only rows that match on both sides.
- LEFT JOIN: every row from the left table; NULLs where there's no match on the right.
- RIGHT JOIN: the mirror of LEFT. FULL OUTER JOIN: all rows from both sides.
- Below, Finance has no employees, so it only appears with a LEFT JOIN from departments.

```sql
SELECT d.dept_name, COUNT(e.emp_id) AS employees
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC;
```

SQL

OUTPUT ✓ RAN ON THE SAMPLE TABLES

dept_name	employees
Engineering	4
Sales	3
HR	2
Finance	0

Common mistake Using COUNT(*) instead of COUNT(e.emp_id) — Finance would wrongly show 1.

Interviewer may ask next What happens to a LEFT JOIN if you add a WHERE condition on the right table's column? (It behaves like an INNER JOIN — put the condition in ON instead.)

07 Find customers who have never placed an order.

Joins Easy

WHAT THE INTERVIEWER IS CHECKING

The anti-join pattern.

MODEL ANSWER

- LEFT JOIN orders and keep rows where the order side is NULL.
- NOT EXISTS works too and handles NULLs safely (NOT IN can break if the subquery returns a NULL).

```sql
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

name
Sameer

Common mistake Using NOT IN (SELECT customer_id FROM orders) when customer_id can be NULL — it returns nothing.

Interviewer may ask next Rewrite it with NOT EXISTS.

08 Find employees who earn more than their manager.

Joins Medium

WHAT THE INTERVIEWER IS CHECKING

Self-join on the same table with two aliases.

MODEL ANSWER

- Join employees to itself: one copy as the employee, one as the manager, linked by manager_id = emp_id.
- Then compare the two salaries.

```sql
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

SQL

OUTPUT ✓ RAN ON THE SAMPLE TABLES

employee	salary	manager	manager_salary
Priya	160000	Asha	150000

Common mistake Forgetting the aliases and getting an 'ambiguous column' error.

Interviewer may ask next List every employee with their manager's name, including those without a manager.

Ranking & Top-N

09 Find the second highest salary.

Ranking Easy

WHAT THE INTERVIEWER IS CHECKING

Handling ties — the classic question.

MODEL ANSWER

- Take the highest salary that is lower than the overall maximum.
- This handles ties at the top correctly (two people on the max salary won't break it).

```sql
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

SQL

OUTPUT ✓ RAN ON THE SAMPLE TABLES

second_highest
150000

Common mistake ORDER BY salary DESC LIMIT 1 OFFSET 1 — wrong when two people share the top salary.

Interviewer may ask next What should it return if every employee has the same salary? (NULL — this query already does that.)

10 Find the Nth highest salary (say N = 3).

Ranking Medium

WHAT THE INTERVIEWER IS CHECKING

Generalising with DENSE_RANK.

MODEL ANSWER

- DENSE_RANK() gives equal salaries the same rank with no gaps, so rank N is the Nth distinct salary.
- Wrap it in a CTE because window functions can't be filtered in WHERE directly.

```sql
WITH ranked AS (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT DISTINCT salary FROM ranked WHERE rnk = 3;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

salary
120000

Common mistake Using ROW_NUMBER — duplicates (two people on 90,000) get different numbers.

Interviewer may ask next Turn this into a reusable query/function that takes N as a parameter.

11 ROW_NUMBER vs RANK vs DENSE_RANK?

Window functions · Easy

WHAT THE INTERVIEWER IS CHECKING

Tie handling — easiest way to show you've used window functions.

MODEL ANSWER

- ROW_NUMBER: always unique (1, 2, 3, 4) even for ties.
- RANK: ties share a rank and the next rank skips (1, 2, 2, 4).
- DENSE_RANK: ties share a rank with no gap (1, 2, 2, 3).

```sql
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY salary DESC;
```

SQL

OUTPUT ✓ RAN ON THE SAMPLE TABLES

name	salary	row_num	rnk	dense_rnk
Priya	160000	1	1	1
Asha	150000	2	2	2
Karan	120000	3	3	3
Rahul	90000	4	4	4
Arjun	90000	5	4	4
Sneha	70000	6	6	5
Vikram	70000	7	6	5

Common mistake Saying RANK and DENSE_RANK are the same.

Interviewer may ask next Which one would you use to pick exactly one row per customer? (ROW_NUMBER.)

12 Get the top 2 earners in each department.

Top-N per group · Medium

WHAT THE INTERVIEWER IS CHECKING

PARTITION BY — very common in analyst rounds.

MODEL ANSWER

- PARTITION BY dept_id restarts the ranking for every department.
- Filter on the rank in an outer query. Use DENSE_RANK if ties should both be included.

```sql
WITH ranked AS (
  SELECT d.dept_name, e.name, e.salary,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk
  FROM employees e JOIN departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary, rnk
FROM ranked
WHERE rnk <= 2
ORDER BY dept_name, rnk;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

dept_name	name	salary	rnk
Engineering	Priya	160000	1
Engineering	Asha	150000	2
HR	Meera	65000	1
HR	Divya	48000	2
Sales	Karan	120000	1
Sales	Sneha	70000	2
Sales	Vikram	70000	2

Common mistake Forgetting PARTITION BY and getting the top 2 of the whole company.

Interviewer may ask next Sales has two people on 70,000 — how does the output change with ROW_NUMBER?

13 Show the highest-paid employee in every department.

Top-N per group · Medium

WHAT THE INTERVIEWER IS CHECKING

Getting the row (name), not just the MAX value.

MODEL ANSWER

- GROUP BY with MAX gives the top salary but loses the name.
- Join back to that result, or use a window function as below.

```sql
SELECT dept_id, name, salary
FROM (
  SELECT dept_id, name, salary,
         MAX(salary) OVER (PARTITION BY dept_id) AS dept_max
  FROM employees
) t
WHERE salary = dept_max
ORDER BY dept_id;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

dept_id	name	salary
1	Priya	160000
2	Karan	120000
3	Meera	65000

Common mistake SELECT dept_id, name, MAX(salary) ... GROUP BY dept_id — invalid or random names in many databases.

Interviewer may ask next Solve it with a correlated subquery instead.

Data quality

14 How do you find duplicate records?

Data quality · Easy

WHAT THE INTERVIEWER IS CHECKING

GROUP BY + HAVING COUNT(*) > 1 on the columns that define a duplicate.

MODEL ANSWER

- Decide which columns define 'the same record', group by them, and keep groups with more than one row.
- Here two orders from customer 103 on different dates have the same amount — we check (customer_id, amount).

```sql
SELECT customer_id, amount, COUNT(*) AS times
FROM orders
GROUP BY customer_id, amount
HAVING COUNT(*) > 1;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

customer_id	amount	times
103	400	2

Common mistake Grouping by the primary key — every row is unique by definition, so nothing shows up.

Interviewer may ask next How would you see the full duplicate rows, not just the groups?

15 Remove duplicates but keep one copy of each.

Data quality · Medium

WHAT THE INTERVIEWER IS CHECKING

ROW_NUMBER to mark extra copies — safe to show as a SELECT first.

MODEL ANSWER

- Number rows within each duplicate group; row 1 is the one to keep, anything above 1 is an extra copy.
- Always SELECT the rows first, then turn it into a DELETE ... WHERE order_id IN (...).

```sql
WITH numbered AS (
  SELECT order_id, customer_id, amount,
         ROW_NUMBER() OVER (PARTITION BY customer_id, amount ORDER BY order_id) AS rn
  FROM orders
)
SELECT order_id AS to_delete FROM numbered WHERE rn > 1;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

to_delete
8

Common mistake Deleting without checking first, or using DISTINCT in a SELECT and thinking the table changed.

Interviewer may ask next Write the final DELETE statement.

Analytics with window functions

16 Calculate a running total of sales by date.

Window functions · Medium

WHAT THE INTERVIEWER IS CHECKING

SUM() OVER (ORDER BY ...).

MODEL ANSWER

- A window SUM ordered by date adds up every order up to the current row.
- Add PARTITION BY customer_id to restart the total for each customer.

```sql
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders
ORDER BY order_date;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

order_date	amount	running_total
2026-01-05	1200	1200
2026-01-18	800	2000
2026-02-02	1500	3500
2026-02-11	400	3900
2026-03-09	700	4600
2026-03-15	2200	6800
2026-03-28	950	7750
2026-03-30	400	8150

Common mistake Trying to do it with GROUP BY — that collapses the rows.

Interviewer may ask next Show a 3-order moving average instead (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW).

17 Show month-over-month revenue change.

Window functions · Medium

WHAT THE INTERVIEWER IS CHECKING

Aggregate first, then LAG().

MODEL ANSWER

- Step 1: total revenue per month. Step 2: LAG() pulls the previous month's total onto the current row.
- SUBSTR(order_date, 1, 7) gives 'YYYY-MM' here; in MySQL use DATE_FORMAT, in Postgres DATE_TRUNC.

```sql
WITH monthly AS (
  SELECT SUBSTR(order_date, 1, 7) AS month, SUM(amount) AS revenue
  FROM orders
  GROUP BY SUBSTR(order_date, 1, 7)
)
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly
ORDER BY month;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

month	revenue	change
2026-01	2000	NULL
2026-02	1900	-100
2026-03	4250	2350

Common mistake Applying LAG on the raw orders instead of the monthly totals.

Interviewer may ask next Show the change as a percentage, rounded to 1 decimal.

18 What share of total revenue does each customer contribute?

Analytics Medium

WHAT THE INTERVIEWER IS CHECKING

Combining GROUP BY with a window over the whole result.

MODEL ANSWER

- Sum per customer, then divide by SUM(...) OVER () — an empty OVER() means 'all rows'.
- Multiply by 100.0 (not 100) to avoid integer division.

```sql
SELECT c.name,
       SUM(o.amount) AS revenue,
       ROUND(100.0 * SUM(o.amount) / SUM(SUM(o.amount)) OVER (), 1) AS pct_of_total
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY revenue DESC;
```

SQL

OUTPUT ✓ RAN ON THE SAMPLE TABLES

name	revenue	pct_of_total
Ravi	3400	41.7
Pooja	2200	27.0
Neha	1750	21.5
Imran	800	9.8

Common mistake Integer division — 2200 / 8150 gives 0 in some databases.

Interviewer may ask next Return only the customers that make up the top 80% of revenue.

19 Find customers whose average order value is above the overall average.

Subqueries Medium

WHAT THE INTERVIEWER IS CHECKING

Subquery inside HAVING.

MODEL ANSWER

- Compare each customer's AVG(amount) with the overall AVG(amount) from a subquery.

```sql
SELECT customer_id, ROUND(AVG(amount), 1) AS avg_order
FROM orders
GROUP BY customer_id
HAVING AVG(amount) > (SELECT AVG(amount) FROM orders);
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

customer_id	avg_order
101	1133.3
104	2200.0

Common mistake Comparing against the average of customer averages (a different number).

Interviewer may ask next Rewrite it with a CTE for readability.

20 Group employees into salary bands and count each band.

CASE WHEN · Easy

WHAT THE INTERVIEWER IS CHECKING

CASE for bucketing — used constantly in reporting.

MODEL ANSWER

- CASE WHEN evaluates conditions top to bottom and returns the first match.
- Group by the same CASE expression (or wrap it in a subquery).

```sql
SELECT CASE
         WHEN salary >= 120000 THEN 'High'
         WHEN salary >= 70000  THEN 'Mid'
         ELSE 'Entry'
       END AS band,
       COUNT(*) AS employees
FROM employees
GROUP BY band
ORDER BY employees DESC;
```

OUTPUT ✓ RAN ON THE SAMPLE TABLES

band	employees
Mid	4
High	3
Entry	2

Common mistake Putting the conditions in the wrong order — 'salary >= 70000' first would swallow the High band.

Interviewer may ask next Pivot it: one row per department, with a column per band (SUM(CASE WHEN ...)).

BEFORE YOUR INTERVIEW

Revision checklist

Tick a question only when you can answer it out loud in under two minutes and give one real example.

- [ ] 01 What is the difference between WHERE and HAVING?
- [ ] 02 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)?
- [ ] 03 How do you handle NULLs? Why doesn't `manager_id = NULL` work?
- [ ] 04 UNION vs UNION ALL?
- [ ] 05 In what order does SQL actually execute a query?
- [ ] 06 Explain INNER, LEFT, RIGHT and FULL OUTER JOIN.
- [ ] 07 Find customers who have never placed an order.
- [ ] 08 Find employees who earn more than their manager.
- [ ] 09 Find the second highest salary.
- [ ] 10 Find the Nth highest salary (say N = 3).
- [ ] 11 ROW_NUMBER vs RANK vs DENSE_RANK?
- [ ] 12 Get the top 2 earners in each department.
- [ ] 13 Show the highest-paid employee in every department.
- [ ] 14 How do you find duplicate records?
- [ ] 15 Remove duplicates but keep one copy of each.
- [ ] 16 Calculate a running total of sales by date.
- [ ] 17 Show month-over-month revenue change.
- [ ] 18 What share of total revenue does each customer contribute?
- [ ] 19 Find customers whose average order value is above the overall average.
- [ ] 20 Group employees into salary bands and count each band.

Think out loud

Say your approach before you write anything. Interviewers score your reasoning, not just the final answer.

Connect it to the business

End answers with what the number means: “so we should focus on Bengaluru”, “so the model is safe to launch”.

Know your own projects

Expect “where did you use this?” after almost every question. Keep one project story ready per topic.

Say what you'd check

Mention nulls, duplicates, leakage or edge cases before being asked — it signals real experience.

Prepared the questions?
Now land the interview.

FirstOffer lists fresh fresher jobs, internships and off-campus drives every day — with direct HR emails, Google Forms and official apply links. No middlemen.

firstoffer.online

Daily job alerts on our WhatsApp Channel · Questions vary by company and role; understand the concepts, don't just memorise answers.